

Research Computing Documentation

Overview ▼

Tutorials ▲

[ColdFront Tutorial \(coldfront_tutorial.html\)](#)

[Linux & Bash Tutorial \(bash_tutorial.html\)](#)

[SSH Tutorial \(ssh_tutorial.html\)](#)

[Slurm Tutorial 1: Getting Started \(slurm_quick_start_tutorial.html\)](#)

[Slurm Tutorial 2: Scaling Up \(slurm_tutorial_2.html\)](#)

[Software Tutorial \(software_tutorial.html\)](#)

[Storage Tutorial \(storage_tutorial.html\)](#)

[Globus Tutorial \(globus_tutorial.html\)](#)

Quick Reference ▼

Misc Instructions ▼

Resources & Links ▼

SSH Tutorial

Table of Contents

- [1 - Key Generation](#)
 - [1.1 - Linux, Mac, and Windows 10 \(or Newer\)](#)
 - [1.2 - Windows 7 \(or Older\)](#)
- [2 - Setting Up Your SSH Key on the Cluster](#)
 - [2.1 - Linux, Mac, and Windows 10 \(or Newer\)](#)
 - [2.2 - Windows 7 \(or Older\)](#)
- [3 - Automatically Entering Your SSH Key Passphrase](#)
 - [3.1 - Linux, Mac, and Windows 10 \(or Newer\)](#)
 - [3.2 - Windows 7 \(or Older\)](#)
- [4 - Removing Offending Keys from the known_hosts File](#)
 - [4.1 - Linux, Mac, and Window 10 \(or Newer\)](#)
 - [4.2 - Windows 7 \(or Older\)](#)
- [5 - Additional Login Options](#)

The [Secure SHell \(SSH\) protocol](#)  is used to securely access remote servers using a public key and private key. The public key acts like a lock—anyone can see it—but only the holder of that corresponding private key can unlock it. When connecting to a remote server that has your public key, your identity is verified by checking if your private key (which only you should have) can unlock your public key. Keeping your private key secure is crucial, as anyone with access to your private key can impersonate you and access servers that have your public key. SSH authentication is more secure than passwords, and we highly recommend that you use SSH authentication to access the Research Computing cluster.

Warning: Before you try to set up SSH keys, you first need to log into the cluster with your password (and a Duo prompt). Multifactor authentication via Duo is required when you login using your password. If you do not receive a Duo prompt, please [make sure you have Duo set up](#) .

1 - Key Generation

In order to use SSH authentication, you need SSH keys. In this section of the tutorial, we will show you how to generate SSH keys on Windows, Mac, and Linux.

Warning: On `sporcssubmit.rc.rit.edu`, you already have SSH keys. These were created when you first logged into the cluster. **DO NOT** modify these keys or you will not be able to login to cluster nodes. These instructions are for creating SSH keys on your local machine so you can SSH into the cluster.

1.1 - Linux, Mac, and Windows 10 (or Newer)

1. If you are using Windows 10, you will need to install the OpenSSH client before continuing with the following steps.

- Go to **Setting > Apps > Optional Features**.
- Click on **Add a feature**.
- Search for **OpenSSH client** and install it.
- **Note:** Windows 11 has the OpenSSH client installed by default.

2. Check to see if you already have SSH keys. If you see the files `id_rsa` and `id_rsa.pub`, you already have SSH keys, and you can skip to Section 2 of this tutorial.

- **Linux:** Run `ls /home/<username>/.ssh/`
- **Mac:** Run `ls /Users/<username>/.ssh/`
- **Windows 10 (or Newer):** Run `ls C:\Users\<username>\.ssh`
- **Note:** Windows may be hiding the file extension (`.pub`) from you; if that's the case, make sure you see two different files called `id_rsa`.

3. If you don't already have SSH keys, you need to generate them.

- **Linux:** Run `ssh-keygen -t rsa`
- **Mac:** Run `ssh-keygen -t rsa`
- **Windows 10 (or Newer):** Run `ssh-keygen -t rsa`

4. You will be prompted for a filename/location to save your keys. Press **Enter** to select the default. **Do not** change the default.

- ```
$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/<username>/.ssh/id_rsa):
```

5. You will be prompted to create a passphrase for your SSH keys. **DO NOT** leave it blank. Make sure you remember this passphrase, as you will need it each time you use your SSH key.

- ```
Enter passphrase (empty for no passphrase):
```

6. You will be asked to re-enter your passphrase. Do so and press **Enter**.

- ```
Enter same passphrase again:
```

7. You will see output similar to this, indicating that your SSH keys have been generated.

- ```
Your identification has been saved in /home/<username>/.ssh/id_rsa
Your public key has been saved in /home/<username>/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:YwOXM1rI+idGlqftABaj7gRtvV9rwWcXGm21NeFSVXw <username>@<hostname>
The key's randomart image is:
+---[RSA 3072]-----+
|                      +*|
|      . . .      +.E|
|    o + *    . o +o|
| . o + * o. + o |
|. + = =.S  + .  |
| + . * =oo+ .  |
| o . * ++ .    |
| o  o *..      |
| .    ..o      |
+-----[SHA256]-----+
```

8. Congratulations, you just set up SSH keys! Move on to Section 2 of this tutorial to set your key up on the Research Computing cluster.

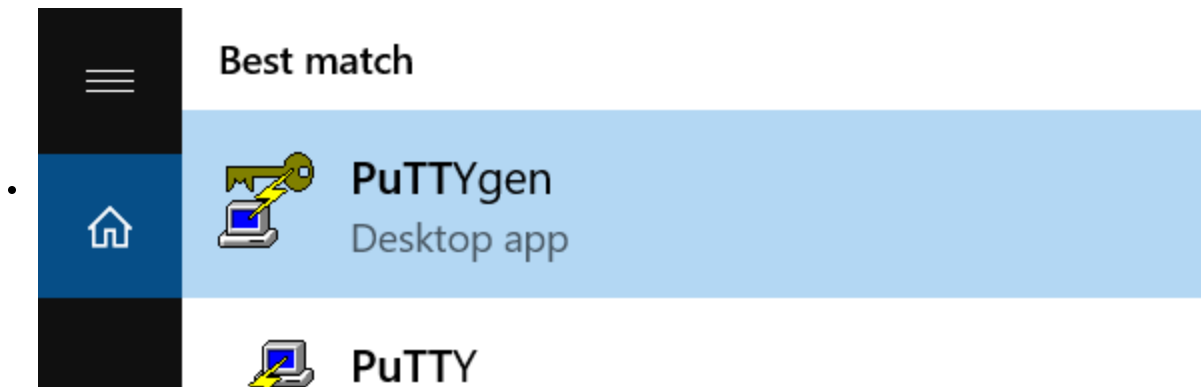
Warning: DO NOT share your SSH **private key** with anyone. Keeping your private key secure is crucial, as anyone with access to your private key can impersonate you and access servers that have your public key.

1.2 - Windows 7 (or Older)

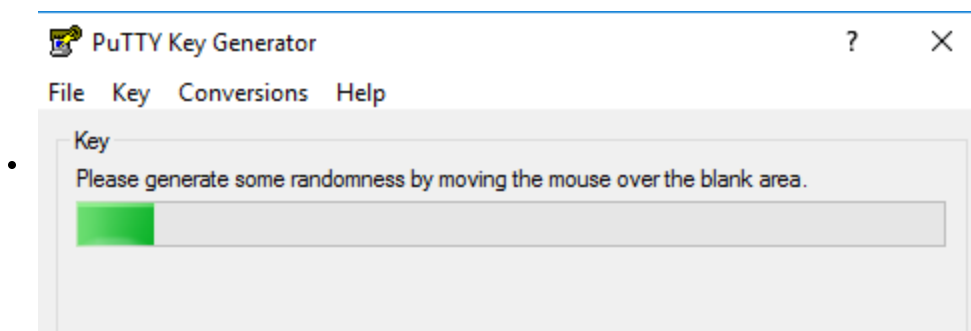
Note: Section 1.2 of this tutorial is included for legacy purposes; it is not being updated or tested. If you run into issues with PuTTY, please consult the [PuTTY Documentation](#).

If you are using Windows 7 or older (not recommended), the OpenSSH client is not available. You will need to use a program called PuTTY to setup and use SSH keys.

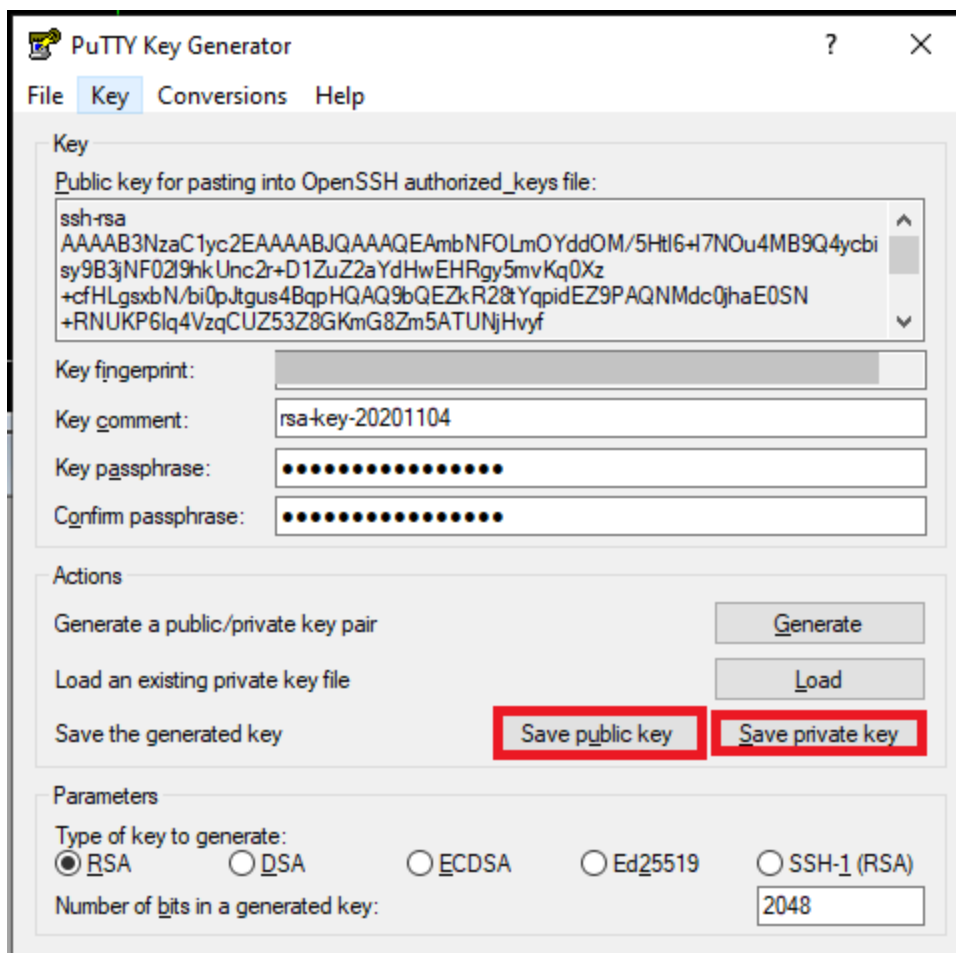
1. Install [PuTTY](#).
2. Open the program **PuTTYgen**. It was installed along with PuTTY.



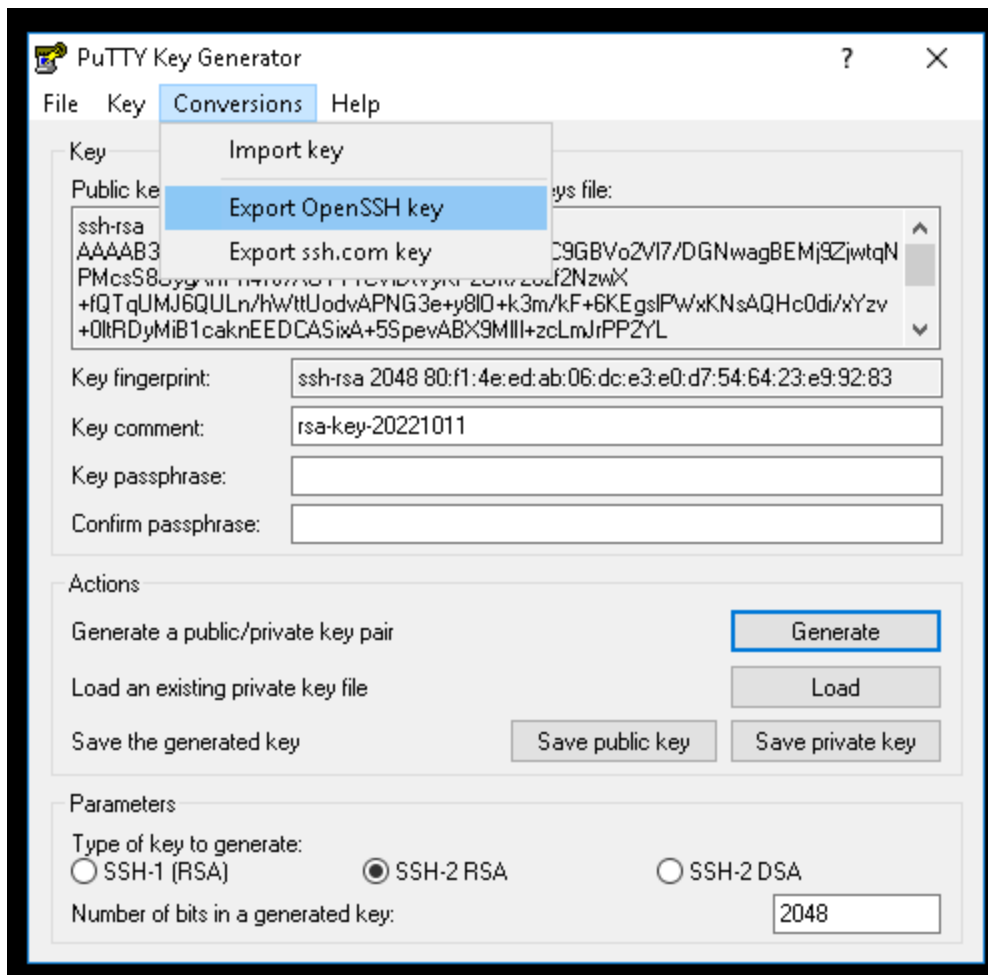
3. Leave all of the settings as they are and click **Generate**. You will be asked to move your mouse around the area to generate randomness.



4. Enter a strong passphrase into the passphrase field. Make sure you remember this passphrase, as you will need it each time you use your SSH key.
5. Save the public key as `id_rsa.pub` in `C:\Users\<username>\.ssh\`.



6. Save the private key as `id_rda.ppk` in `C:\Users\\.ssh\`.
7. Export your OpenSSH key by clicking **Conversions > Export OpenSSH key**.
 - Name this key `id_rsa` and save it in `C:\Users\\.ssh\`.



8. Congratulations, you just set up SSH keys! Move on to Section 2 of this tutorial to set your key up on the Research Computing cluster.

2 - Setting Up Your SSH Key on the Cluster

Now you are ready to copy your SSH public key to the Research Computing cluster!

2.1 - Linux, Mac, and Windows 10 (or Newer)

1. Copy your SSH public key to the cluster.

- **Linux:** Run `ssh-copy-id -i ~/.ssh/id_rsa.pub`
`<rit_username>@sporcsuubmit.rc.rit.edu` **Note:** If `ssh-copy-id` is not available on your computer, you can use this alternative command: `cat ~/.ssh/id_rsa.pub | ssh <rit_username>@sporcsuubmit.rc.rit.edu "cat - >> ~/.ssh/authorized_keys"`
- **Mac:** Run `cat ~/.ssh/id_rsa.pub | ssh <rit_username>@sporcsuubmit.rc.rit.edu "cat >> ~/.ssh/authorized_keys"`
- **Windows 10 (or Newer):** Run `type C:\Users\<username>\.ssh\id_rsa.pub | ssh <rit_username>@sporcsuubmit.rc.rit.edu "cat >> ~/.ssh/authorized_keys"`

2. You may be shown the key fingerprint for `sporcsu` and asked if you still want to install your public key. Type `yes` and press “Enter”.
3. You will be asked for your password. Type your **RIT password** and press “Enter”.
 - **Note:** The terminal will not show your password while typing. This is to prevent others from seeing your password.
 - **Note:** Multifactor authentication via Duo is required when you login using your password. If you do not receive a Duo prompt, please [make sure you have Duo set up](#) .
4. You should see the following message:

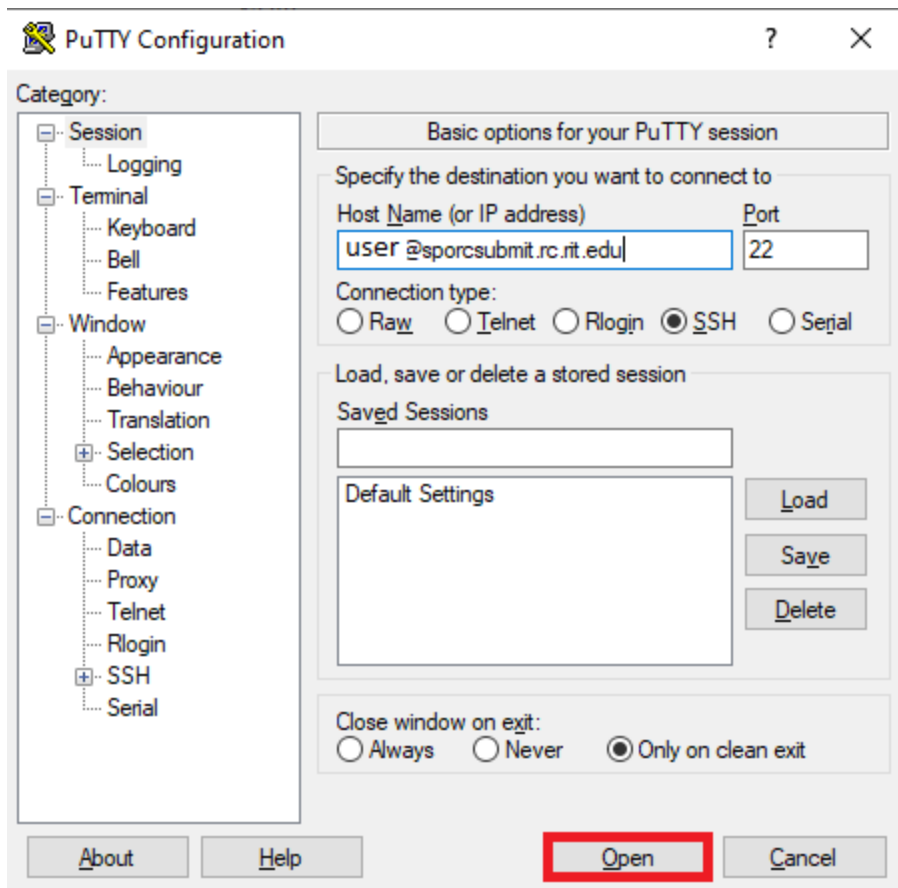
- ```
Number of key(s) added: 1
Now try logging into the machine, with: "ssh <rit_username>@sporcsu
t.edu"
```

5. Now you can login to the cluster using your SSH key: `ssh <rit_username>@sporcsu`  
`t.edu`
6. You will be prompted for your SSH key’s passphrase, which you set up in Section 1.1, Step 5 above.
  - If you do not want to type your SSH key’s passphrase every time you login, you can find details on setting up `ssh-agent` in Section 3 of this tutorial.
7. From now on, you can login to the cluster using your SSH key by running:
  - `$ ssh <rit_username>@sporcsu`  
`t.edu`.

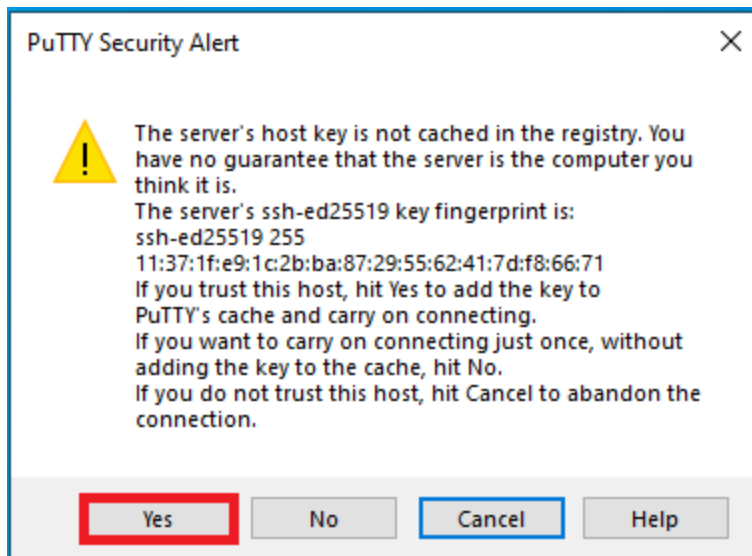
## 2.2 - Windows 7 (or Older)

**Note:** Section 2.2 of this tutorial is included for legacy purposes; it is not being updated or tested. If you run into issues with PuTTY, please consult the [PuTTY Documentation](#) .

1. Open PuTTY.
2. In the “Host Name (or IP address)” field, enter `<rit_username>@sporcsu`  
`t.edu`.



3. If this is your first time connecting, you will be asked to recognize the authenticity of the host. Click “Yes”.

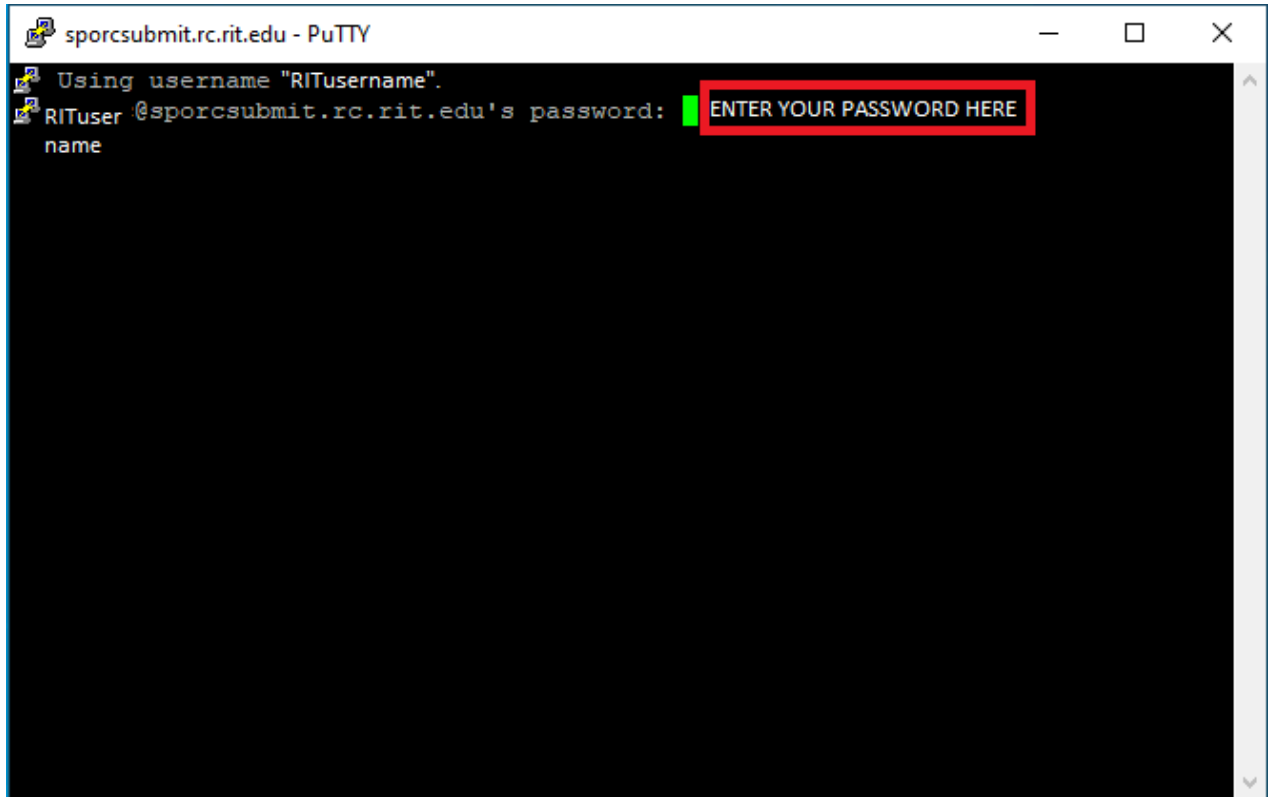


4. Type in your **RIT password**. Press “Enter”.

- **Note:** The terminal will not show the password while typing. This is to prevent others from seeing your password.



- **Note:** Multifactor authentication via Duo is required when you login using your password. If you do not receive a Duo prompt, please make sure you have Duo set up [↗](#).



5. Check to see if you have the file `/home/<username>/.ssh/authorized_keys`:

- ```
$ ls /home/<username>/.ssh/
authorized_keys  id_rsa  id_rsa.pub  known_hosts
```

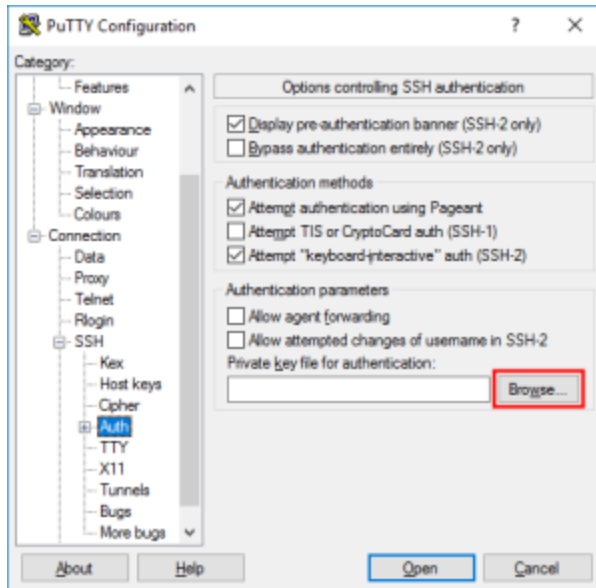
- If you don't, you can create it: `touch /home/<username>/.ssh/authorized_keys`

6. Open the `authorized_keys` file with a text editor. We will use `nano` in this example: `$ nano /home/<username>/.ssh/authorized_keys`.

7. On your **local** machine, copy your **public key**. You can open `id_rsa.pub` with any text editor. It's contents should look similar to this:

- ```
ssh-rsa AAAAB3NzaClyc2EAAAADAQABAAQGC9rYXQ0m/Og9o8KLr8lxcMFgavY0+BCv97Qo10
cF/xQ+m52IYeuuLuW2ZUMy0BhWbVjUT8YORrbILq4ZXYVmMgGP9ES8rlFrcNgkOPGGfqY+Ay8/
o3qhtxnJqUgWAYr4KuUxkl9viP8KZ+mgp3UV68fPiKBq7ClKkEveUmBqXGZHPrgkLkizaU2p5+y4
7WlIJMOz09MYeOQm9AJLDTcCGdabvYou58IBMUjNvqmjB8oXsk/nv+sDx6hrhJkZjIV04aCrkf49
oxloNCtFqd2TrdCNkpBSZwhSgZYubTwTXnP+k+23FpZrxERM+8q7fMVm9VCeHXOXmzIDMupWm8T7B
A/2DO7JB95tqvFMksn+VY0yn5nMMseB2P2mu6D9ZcsFAYyEFtckWm6fBcymbah9CZm6rLbvYu4S
ayVxg5DcVGI8VUOF4tGqgde+gcisAV7bBLNy3T32tcKsOn5/7sbmVeZkr2zVzX1UeUCwEgNZT2Pb
ZO9T72eHuWCP+N1Yab8= <username>@<hostname>
```

- **Note:** If you still have PuTTYgen open, you can also copy your public key from there.
- At the end of the `authorized_keys` file, on a new line, paste your **public key**. Save the file.
    - With `nano`: Type `CTRL + X`, then `y`, then press “Enter”.
  - Close PuTTY.
  - Open PuTTY again and navigate to “Connection” → “SSH” → “Auth”.



- Click “Browse” and navigate to your **private key**. Select your private key.
- Go back to the “Session” menu and repeat Step 2 of this section.
- If everything worked, you should not be prompted for your password. You will however be prompted for your SSH key passphrase. Type it in and press “Enter”.
  - To circumvent entering your passphrase every time you SSH into a machine, see Section 3.2 of this tutorial for details on using Pageant.
- From now on, you only need to use Steps 12-13 to connect to the cluster.

## 3 - Automatically Entering Your SSH Key Passphrase

If you do not want to type your SSH key passphrase every time you login to the cluster, you can use `ssh-agent`. You must have SSH keys set up before you can use `ssh-agent`.

### 3.1 - Linux, Mac, and Windows 10 (or Newer)

- Start the `ssh-agent` process.
  - **Linux:** Run `eval $(ssh-agent)`
  - **Mac:** Run `eval $(ssh-agent)`

- **Windows 10 (or Newer):**

- ```
Get-Service ssh-agent | Set-Service -StartupType Automatic  
Start-Service ssh-agent
```

2. Provide your SSH key passphrase to `ssh-agent`.

- **Linux:** Run `ssh-add ~/.ssh/id_rsa`
- **Mac:** Run `ssh-add ~/.ssh/id_rsa`
- **Windows 10 (or Newer):** Run `ssh-add C:\Users\<username>\.ssh\id_rsa`

3. Type in your SSH key passphrase and press “Enter”.

4. Now you can login to the cluster using your SSH keys without typing your passphrase.

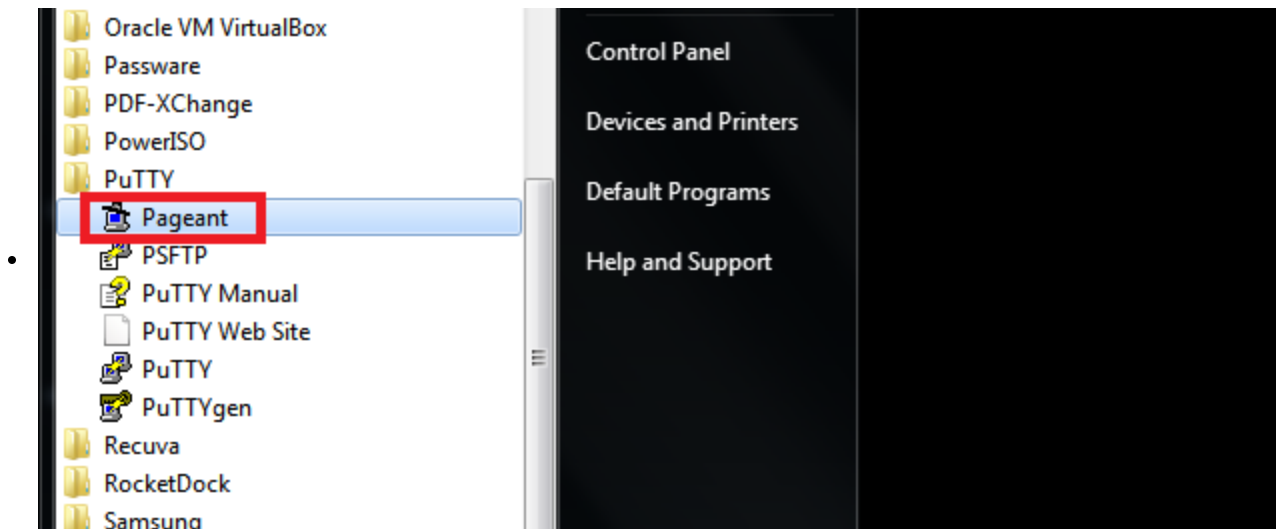
- **Note:** If you close your terminal or reboot your computer, you will have to repeat this process to use `ssh-agent`.

3.2 - Windows 7 (or Older)

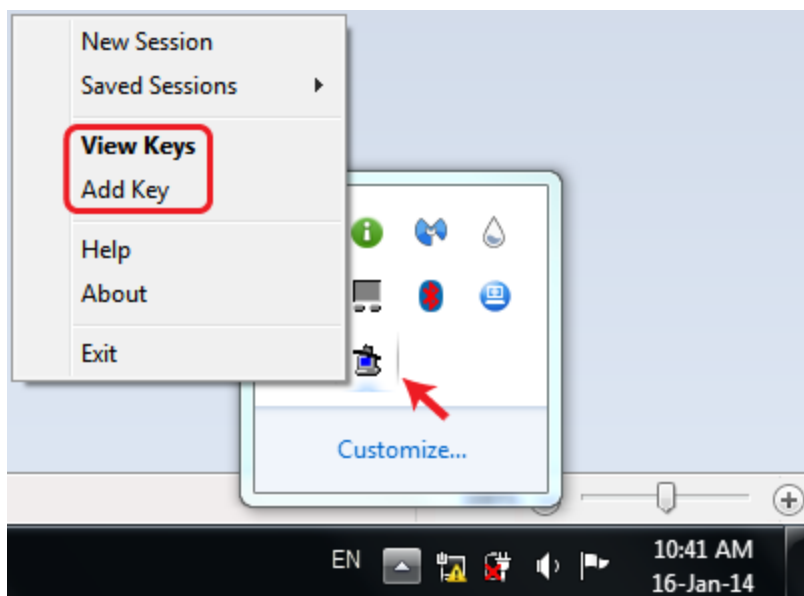
Note: Section 3.2 of this tutorial is included for legacy purposes; it is not being updated or tested. If you run into issues with PuTTY, please consult the [PuTTY Documentation](#) ↗.

Pageant is a tool included with PuTTY that runs as a background process and enters the passphrase for the private keys for you. you must have a key already set up to use Pageant.

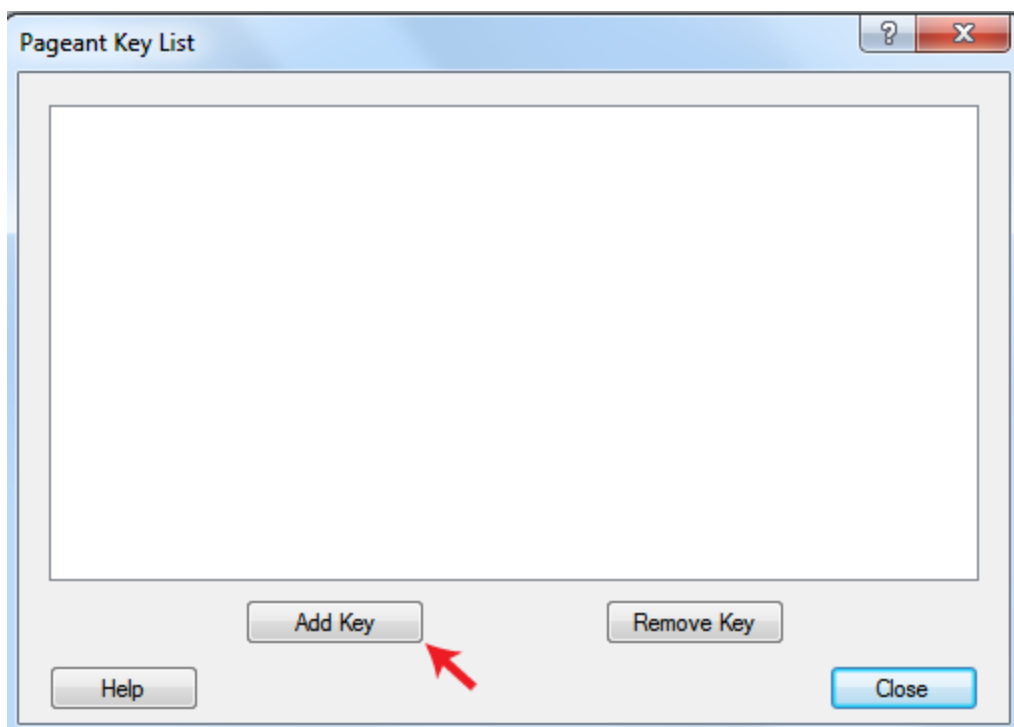
1. Open Pageant.



2. Pageant starts minimized in the system’s tray. Right click on the Pageant icon and a menu will appear:

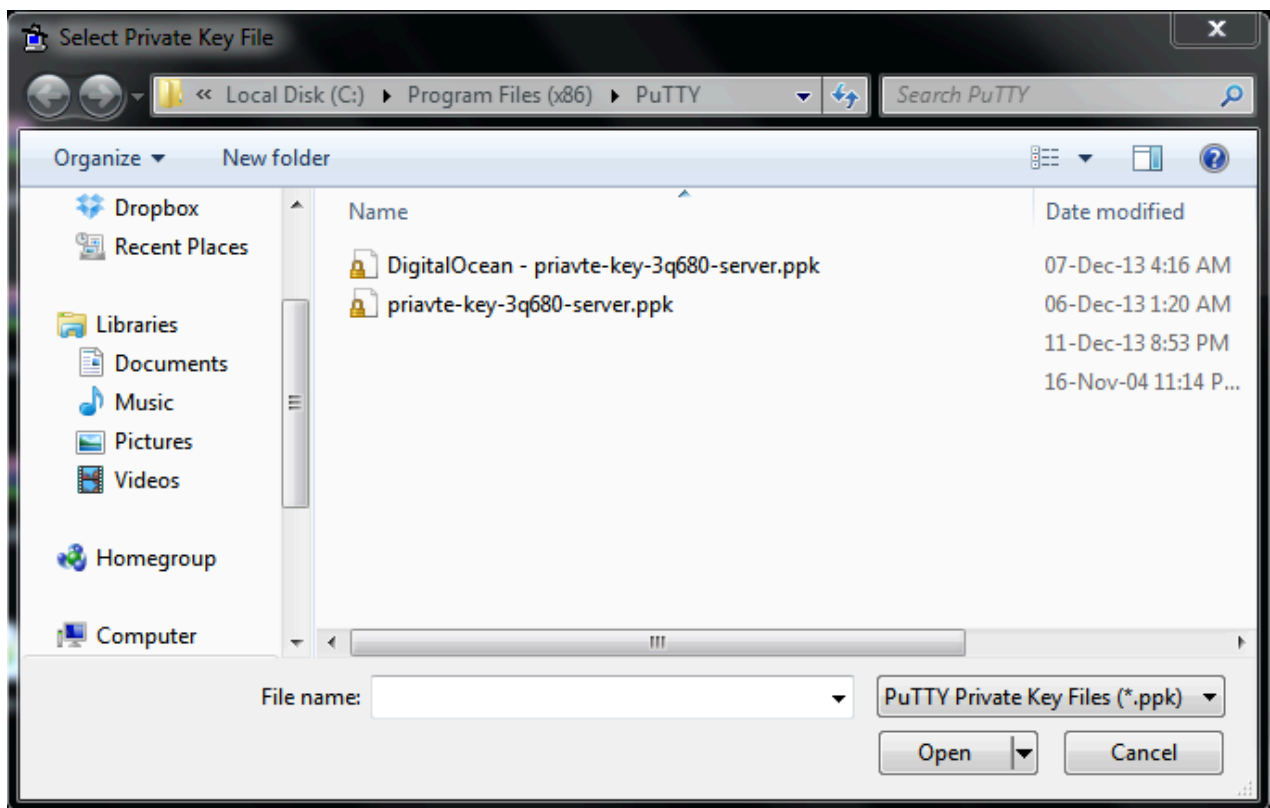


3. Click “View Keys” in the menu that appears.

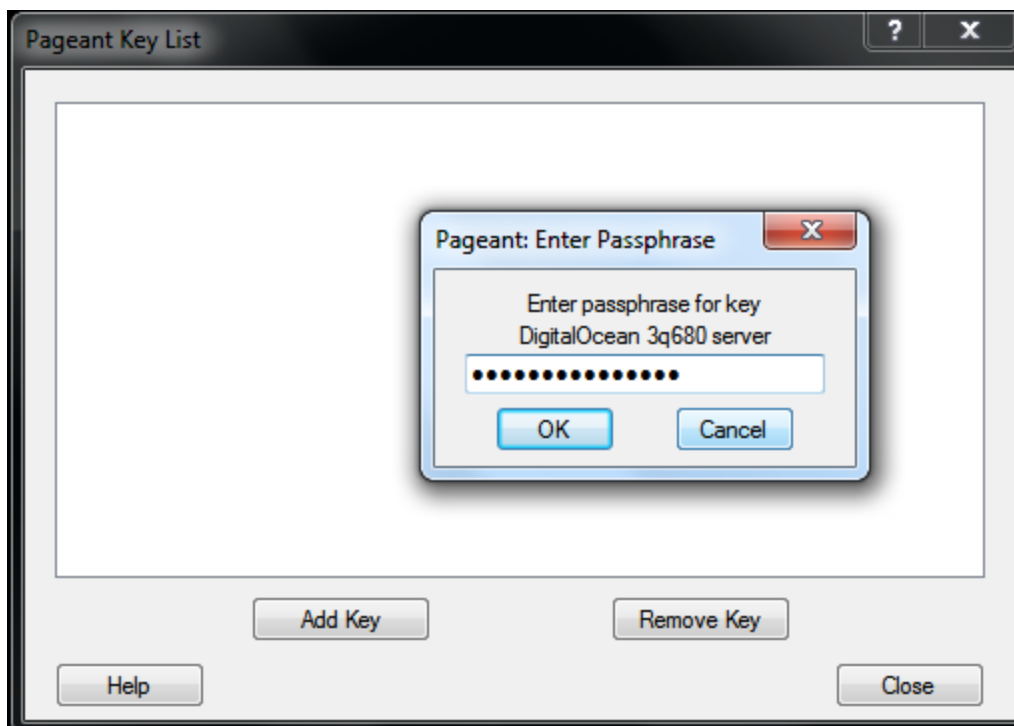


4. Click “Add Key” in the window that appears.

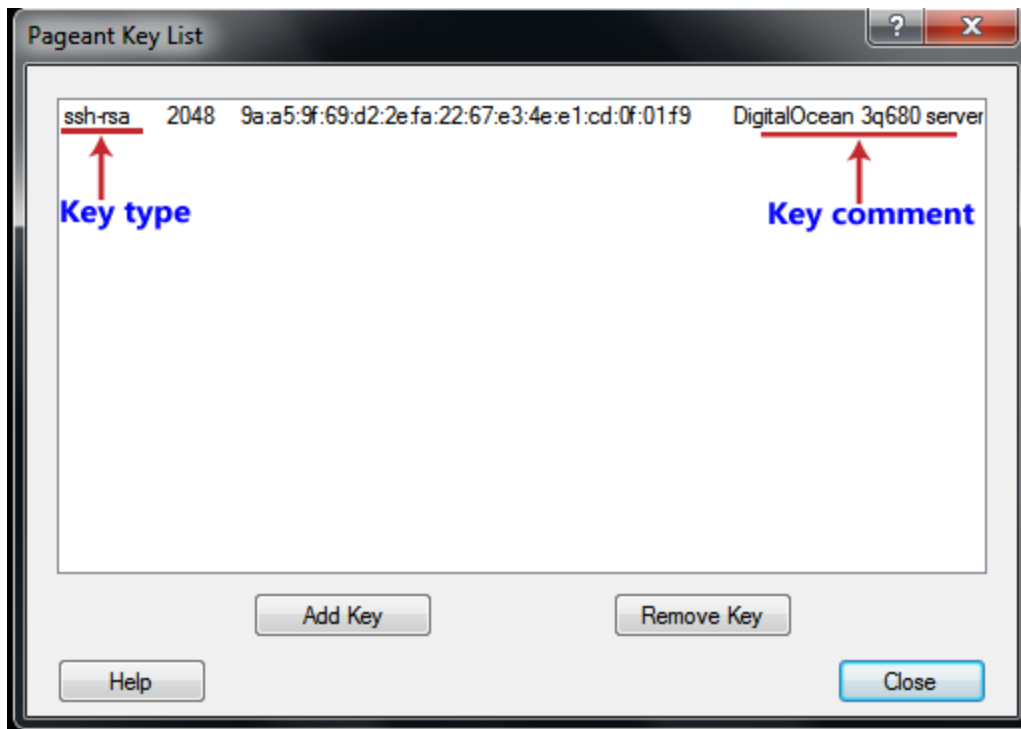
5. Open the key you want to add. It must be a key generated from PuTTY (ending in .ppk).



6. You will be prompted for your passphrase. Type it in and click “Ok”.



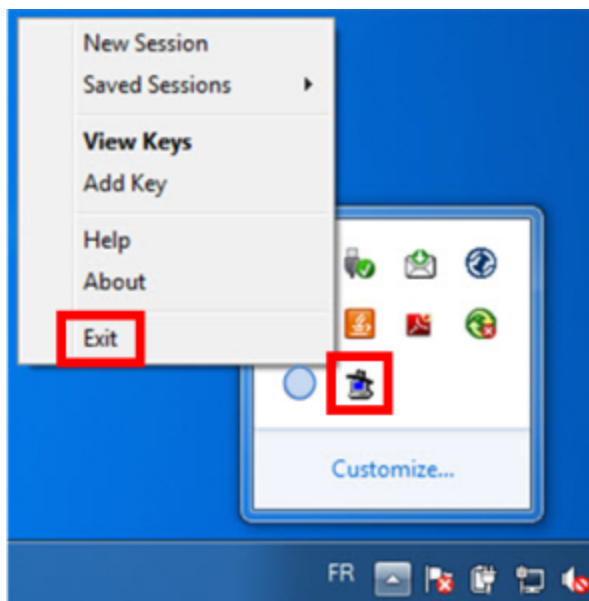
7. Now the key is loaded into Pageant.



8. Now you can connect to the cluster using PuTTY without typing in your SSH key passphrase.

9. Make sure when you are done using SSH (via PuTTY) to connect to the cluster that you close Pageant.

- Each time you open Pageant, you need to repeat this process to add the key again.



4 - Removing Offending Keys from the known_hosts File

After you connect to a **remote** machine, the key you used to connect is stored in the `known_hosts` file on your **local** machine. The purpose of this file is to verify the identity of the remote machines you are connecting to.

Occasionally, you might see an error like this when you try to connect using SSH:

```

@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@      WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!      @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that the RSA host key has just been changed.
The fingerprint for the RSA key sent by the remote host is
5c:9b:16:56:a6:cd:11:10:3a:cd:1b:a2:91:cd:e5:1c.
Please contact your system administrator.
Add correct host key in /home/user/.ssh/known_hosts to get rid of this message.
Offending key in /home/user/.ssh/known_hosts:1
RSA host key for ras.mydomain.com has changed and you have requested strict checking.
Host key verification failed.

```

This error will occur when the fingerprint the host has changed. Check with the administrator of that remote machine to make sure the change is expected. To remove the old fingerprint and login again, you can follow these instructions.

4.1 - Linux, Mac, and Window 10 (or Newer)

1. Remove the key:

- **Linux:** Run `ssh-keygen -R <hostname>`
- **Mac:** Run `ssh-keygen -R <hostname>`
- **Windows 10 (or Newer):** Run `ssh-keygen -R <hostname>`

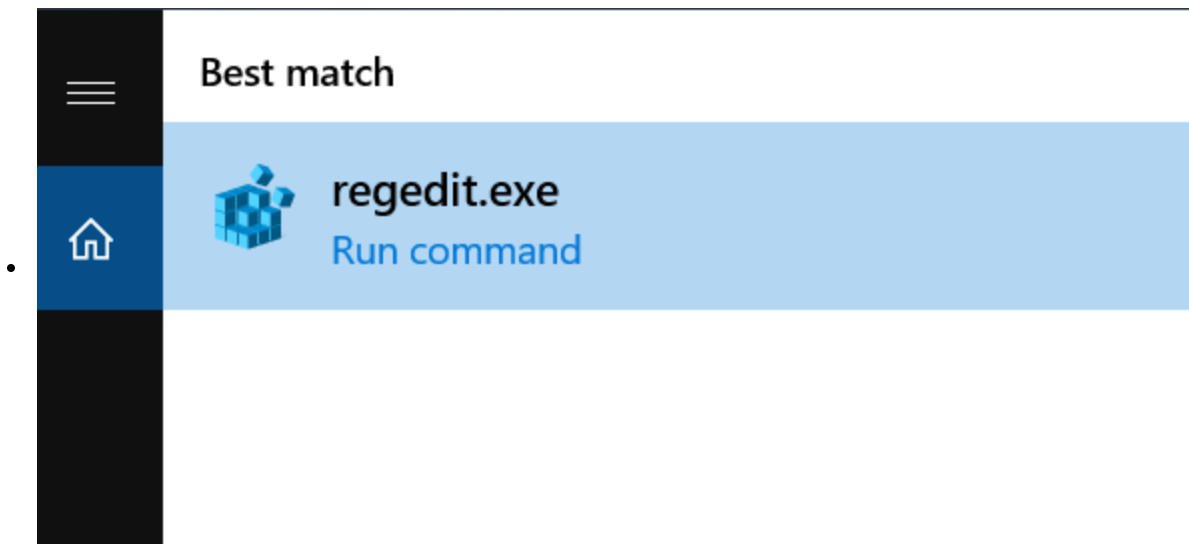
2. Login like you normally would and access the new fingerprint by typing `yes` and pressing “Enter”.

4.2 - Windows 7 (or Older)

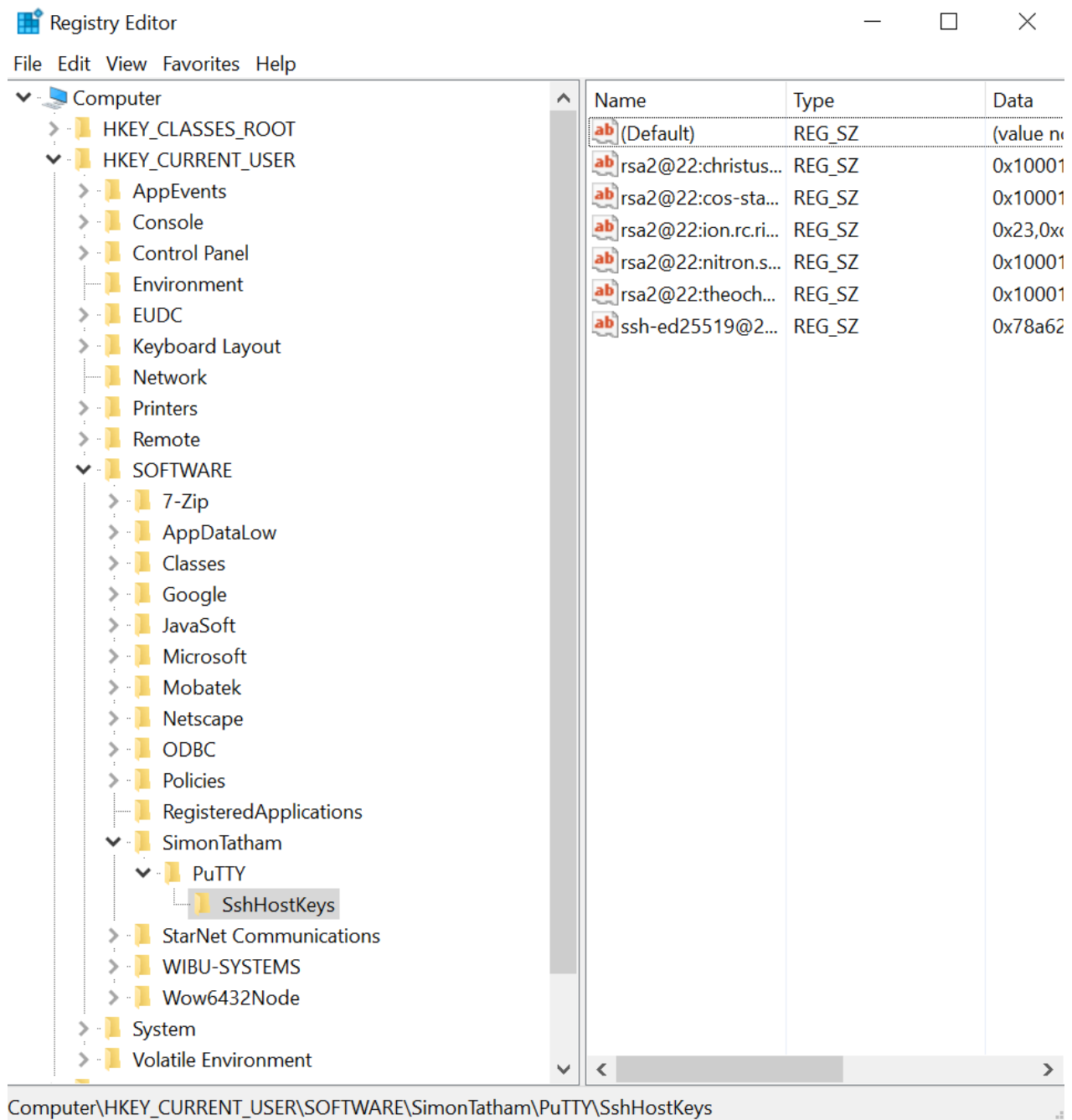
Note: Section 4.2 of this tutorial is included for legacy purposes; it is not being updated or tested. If you run into issues with PuTTY, please consult the [PuTTY Documentation](#) .

PuTTY does not create a `known_hosts` file, but we can still remove offending keys this way:

1. Open `regedit.exe`.



2. Navigate to “HKEY_CURRENT_USER/SOFTWARE/SimonTatham/PuTTY/SshHostKeys”.



3. Right click on the offending key and click "Delete".

Name	Type	Data
 (Default)	REG_SZ	(value n
 rsa2@22:christus...	REG_SZ	0x1000'
 rsa2@22:cos.cta...	REG_SZ	0x1000'
 rsa2@22		x23,0xi
 rsa2@22		x1000'
 rsa2@22		x1000'
 ssh-ed25		x78a62

Modify...

Modify Binary Data...

Delete

Rename

5 - Additional Login Options

When logging into a remote server, you can use additional flags to enhance your SSH session:

- `ssh -X`: Enables X11 forwarding, allowing you to run graphical applications over SSH.
- `ssh -A`: Enables authentication agent forwarding, allowing the remote server to use your local SSH keys; **do not** use this unless you know why you are using it.
- `ssh -vvv`: Provides verbose output, showing detailed information about the connection process; this may be using for debugging.

Help: If there are any further questions, or there is an issue with the documentation, please **submit a ticket** [↗](#) or contact us on Slack for additional assistance.

Tags: [instructions \(tag_instructions.html\)](#) [ssh \(tag_ssh.html\)](#)

